ASSIGNMENT – 2

SOFTWARE TESTING AND QUALITY ASSURANCE

NAME: SRISHTI MALIK ROLLNO. :253028081

Q1. Testing Strategy for an Online Examination Management System

An Online Examination Management System allows students to register, select subjects, pay examination fees, download admit cards and view results. A complete testing strategy checks that every feature works correctly and protects student information.

1. Testing fundamentals

- Testing should start early, from the requirement stage, because errors are cheaper to fix before coding.
- Testing checks whether the system follows the requirements and helps find defects. It cannot prove that the software has no defects at all.
- Testing should cover normal cases, invalid inputs, boundary values and situations where services fail.
- High-risk features such as payments, student eligibility and result calculation should receive more testing.

2. Types of testing and their use

Testing type	How it is used	Why it is needed
Unit testing	Test individual functions such as fee calculation, subject selection and grade calculation.	Finds errors in small parts of the program.
Integration testing	Test communication between registration, payment gateway, admit-card service and result database.	Finds problems when modules exchange data.
System testing	Test the complete system from student registration to result viewing.	Checks whether the whole system works correctly.
Functional testing	Check each feature against its requirement.	Confirms that required functions are available.
Performance testing	Test the system when many students register or view results at the same time.	Checks speed and stability under heavy use.
Security testing	Check login, permissions, payment protection and result privacy.	Prevents unauthorized access and data leakage.
Regression testing	Repeat important tests after a change or bug fix.	Checks that new changes have not broken old features.
Acceptance testing	Students and examination staff test realistic tasks.	Confirms that the system meets users' needs.

3. Static testing, dynamic testing, verification and validation

- Static testing: Requirements, design documents and code are reviewed without running the program. It can find unclear eligibility rules, missing payment failure handling and weak access control.
- Dynamic testing: The software is executed with test data. For example, test successful and failed payments, invalid registration details, admit-card downloads and result access.

- Verification: It checks, “Are we building the product correctly?” Reviews and inspections confirm that documents and code follow specifications.
- Validation: It checks, “Are we building the correct product?” Real users check whether the system meets their needs.

Conclusion: These activities are needed to detect defects early, protect student data, calculate results correctly and make the examination process reliable.

Q2. Static Testing of an Online Banking Application

Static testing checks requirements, design documents, source code and database definitions without executing the banking application. It can detect many defects before dynamic testing begins.

Area	Possible defect	Example
Requirements	Unclear transfer limits or beneficiary rules.	Developers may implement different daily transfer limits.
Security	Missing authentication, authorization or session timeout rules.	A user may gain access to another customer’s account.
Input validation	No checks for negative or zero transfer amounts.	An invalid amount may be accepted by the program.
Money calculations	Incorrect rounding or unsafe numeric data types.	The transferred amount may be calculated incorrectly.
Database	Missing transaction controls or unique transaction IDs.	Money may be debited without being credited correctly.
Error handling	Errors are ignored or internal details are displayed.	The system may fail silently or expose sensitive information.
Code quality	Duplicated code and complicated conditions.	The same defect may exist in multiple places.
Interfaces	Different modules use inconsistent data formats.	The banking screen and transfer service may interpret a response differently.

Static testing reduces cost because defects are found before setting up test data and running the application. It also helps testers prepare better test cases. However, it cannot replace dynamic testing, which is needed to check actual execution, integration, timing and runtime behavior.

Q3. Formal Review Process for a 2,000-Line Code Module

The module has inconsistent names, duplicated code, few comments and complex conditions. A formal review can find these problems in an organized way.

Participants

- Author: The developer who wrote the code and will correct the defects.
- Moderator: Organizes the review and ensures that the process is followed.
- Reviewers: Other developers, a tester and, if needed, a security or subject expert.
- Recorder: Writes down defects, decisions and action items.

Review stages

1. Planning: Select the code, define the review goal, choose participants and share the coding standards.
2. Overview: The author explains the purpose, design and important functions of the module.
3. Preparation: Reviewers read the code separately and use a checklist to find issues.
4. Review meeting: Discuss each issue with its location and evidence. Focus on finding defects rather than blaming the author.
5. Correction: The author fixes agreed defects and adds or updates tests if needed.
6. Follow-up: The moderator checks the fixes and confirms that important issues are closed.

Defect classification

Severity	Example
Critical	A serious security weakness or logic error that can cause major damage.
Major	Incorrect processing, unsafe error handling or serious violation of a requirement.
Minor	Naming inconsistency, missing helpful comment or small readability issue.
Suggestion	An optional improvement that is not a confirmed defect.

Checklist and documentation

- Check naming, formatting, duplicated logic, function size, complex conditions, comments, input validation, security, exception handling and resource cleanup.
- Record a defect ID, file and line number, description, severity, assigned person, status and verification result.
- Keep the checklist, attendance, meeting notes, defect list and final review summary.

The review is complete when required fixes have been verified and remaining issues are documented and assigned.

Q4. Peer Review and Formal Review

Peer review	Formal review
Usually a simpler review by one or more colleagues.	Follows a planned and documented process.
Often takes place through a pull request or code review tool.	Usually defines roles, preparation, meeting records and follow-up.
Suitable for frequent day-to-day code changes.	Useful for critical modules that need stronger control and traceability.

Example: Five developers are working on an e-commerce application. One developer creates the checkout or payment module. Two colleagues review the code before it is merged into the main project.

7. Identify defects: Reviewers check payment logic, input validation, access permissions, duplicate requests and error handling.
8. Record defects: Each issue is written as a clear comment with the relevant line or function.
9. Discuss defects: The author and reviewers clarify the issue and agree on the required correction.
10. Correct defects: The author changes the code and adds or updates tests.
11. Re-verify: A reviewer checks the new code and test results before approving the change.

A formal review uses more documentation and defined steps. A peer review can be lighter, but both should ensure that defects are corrected and verified.

Q5. Coding-Review Checklist with Reasons

Criterion	What to check	Why it is important
Naming conventions	Use clear and consistent names for variables, functions and classes.	Makes code easier to understand.
Formatting	Use consistent indentation, spacing and braces.	Makes logic easier to read.
Code structure	Keep modules focused on one main responsibility.	Makes changes safer.
Function size	Avoid very long functions.	Makes testing and debugging easier.
Duplicate code	Reuse common logic where appropriate.	Prevents inconsistent fixes.
Complex conditions	Simplify deeply nested conditions.	Reduces logic mistakes.
Comments	Explain important decisions and non-obvious logic.	Helps future developers understand the code.
Documentation	Document public functions, assumptions and interfaces.	Makes modules easier to use.
Input validation	Check type, range, length and format of user input.	Rejects invalid or unsafe data.
Authentication	Verify a user's identity before protected operations.	Prevents unauthorized access.
Authorization	Check that a user has permission for each operation.	Protects restricted information and functions.
Sensitive data	Do not hard-code passwords, keys or tokens.	Reduces the risk of credential leakage.
Database queries	Use parameterized queries.	Helps prevent SQL injection.
Output handling	Safely encode untrusted data shown in a browser.	Helps prevent script injection.
Error handling	Show useful errors without exposing internal details.	Improves safety and troubleshooting.
Exception handling	Catch expected exceptions and handle them properly.	Prevents hidden failures.
Logging	Log useful events but never log passwords or secret tokens.	Helps debugging without exposing secrets.
Resource management	Close files, database connections and streams.	Prevents resource leaks.
Performance	Avoid unnecessary work and inefficient database queries.	Improves response time.
Memory use	Avoid storing unlimited data unnecessarily.	Reduces memory problems.
Boundary checks	Test minimum, maximum, empty and overflow values.	Finds off-by-one and range errors.
Return values	Check function results and status codes.	Prevents silent failures.
Testing	Include normal, invalid and boundary test cases.	Provides evidence that the code works.
Concurrency	Check shared data and simultaneous operations.	Prevents inconsistent updates.

Dependencies	Use approved, updated libraries and safe configuration.	Reduces security and compatibility risks.
Maintainability	Keep code readable and follow team standards.	Makes future changes easier.
Accessibility	Make user-facing controls and messages accessible where applicable.	Helps users with different needs.
Regression risk	Re-test existing features after changes.	Finds unintended side effects.

Q6. Equivalence Partitioning for Student Results

Equivalence Partitioning divides input data into groups that should behave similarly. The system accepts marks from 0 to 100 for five subjects.

Assumption: marks must be whole numbers. For the example below, assume a student must score at least 40 in every subject to pass. The actual grade boundaries must follow the university's rules.

Partition	**Test data**	**Expected result**
Valid marks	0, 40, 75, 100	Accept each mark.
Below minimum	-1	Reject the mark.
Above maximum	101	Reject the mark.
Missing mark	Blank for one subject	Show required-field error.
Wrong data type	"abc" or 45.5 if only integers are allowed	Reject the value.
Normal valid record	40, 50, 60, 70, 80	Total 300; percentage 60%; status based on rules.
All zero marks	0, 0, 0, 0, 0	Total 0; percentage 0%; fail under the stated assumption.
All maximum marks	100, 100, 100, 100, 100	Total 500; percentage 100%.
One subject below pass mark	39, 80, 80, 80, 80	Fail if every subject must be passed.

Boundary tests should also check -1, 0, 1, 99, 100 and 101. These cases are useful because errors often occur at the minimum and maximum limits. The selected cases cover valid data, invalid data, missing values, wrong types and important result conditions without testing every possible mark combination.

Q7. Equivalence Partitioning and Boundary Value Analysis for Discounts

The discount rules are: below ₹1,000 = 0%; ₹1,000–₹4,999 = 5%; ₹5,000–₹9,999 = 10%; ₹10,000–₹49,999 = 15%; ₹50,000 or more = 20%.

Test amount	**Expected discount**	**Purpose**
−₹1	Reject	Invalid negative amount.
₹0	0%	Minimum valid amount.
₹999	0%	Just below ₹1,000.
₹1,000	5%	First threshold.
₹1,001	5%	Just above first threshold.
₹4,999	5%	Upper end of 5% group.

₹5,000	10%	Second threshold.
₹5,001	10%	Just above second threshold.
₹9,999	10%	Upper end of 10% group.
₹10,000	15%	Third threshold.
₹10,001	15%	Just above third threshold.
₹49,999	15%	Upper end of 15% group.
₹50,000	20%	Fourth threshold.
₹50,001	20%	Just above fourth threshold.

For systems accepting paise, also test ₹999.99, ₹1,000.01, ₹4,999.99, ₹5,000.01, ₹9,999.99, ₹10,000.01 and ₹49,999.99. These tests can expose wrong comparison operators, missing ranges, overlapping ranges, incorrect percentages and rounding errors. Verify both the discount rate and the final payable amount.

Q8. Test to Pass and Test to Fail for an ATM

Test to Pass checks whether the ATM works correctly with valid inputs. Test to Fail checks whether it handles invalid inputs and failures safely.

Approach / scenario	Condition	Expected result
Pass: normal withdrawal	Valid card and PIN, active account, enough balance and working network.	Cash is dispensed once and the correct amount is deducted.
Pass: minimum amount	Withdraw the minimum permitted amount.	Accept if it follows ATM rules.
Pass: maximum amount	Withdraw the maximum allowed amount with sufficient balance.	Accept and update balance correctly.
Fail: invalid amount	Zero, negative or unsupported denomination amount.	Reject; no money is deducted.
Fail: insufficient balance	Requested amount is greater than available balance.	Decline; no cash is dispensed.
Fail: invalid PIN	Enter an incorrect PIN.	Deny access and update failed attempts according to policy.
Fail: blocked card	Use a blocked or expired card.	Reject the transaction.
Fail: network failure	Network fails before authorization.	Do not dispense cash; handle the transaction safely.
Fail: timeout	The ATM does not receive a response in time.	Show a suitable message and check transaction status.
Recovery after timeout	The bank authorizes but the ATM does not receive confirmation.	Prevent duplicate debit and reconcile or reverse the transaction.
Repeated request	Repeat a withdrawal after an uncertain response.	Do not process the same transaction twice.
Receipt and history	Complete a successful withdrawal.	Receipt and transaction history show the correct amount.

For every failure, check cash dispensing, account balance, transaction record, error message and reversal process. This prevents money loss and duplicate transactions.

Q9. Data Testing for a Hospital Appointment System

Data type	Example	Expected check

Valid data	Valid age, registered doctor, future date and successful payment.	Appointment is created correctly.
Invalid data	Negative age, unknown doctor or past appointment date.	Reject invalid values.
Boundary data	Age at the permitted minimum or maximum.	Apply the documented boundary rule.
Missing data	No age, doctor or appointment date.	Show required-field messages.
Wrong data type	Letters in an age field or invalid date text.	Reject invalid data types.
Duplicate data	Double-click booking or repeat payment submission.	Avoid duplicate bookings and charges.
Inconsistent data	Payment marked successful when the payment service says it failed.	Keep booking and payment records consistent.
Extreme values	Very long text or extremely large age value.	Reject safely without crashing.
Malicious input	SQL-like text, script tags or unexpected characters.	Treat input safely and prevent injection.
Date edge cases	Leap day or appointment at a slot boundary.	Check dates and time slots correctly.
Payment failure	Declined or delayed payment confirmation.	Do not falsely confirm a paid appointment.
Privacy data	Patient details appearing in logs or error messages.	Protect sensitive patient information.

Testing should cover forms, APIs, database rules, booking conflicts, payment responses and recovery after network failures. Use fictional patient records in testing rather than real patient information.

Q10. State Transition Testing for a Bank Account

State Transition Testing checks how an account changes from one state to another after an event. The states in the question are New Account, Active, Suspended and Closed.

Current state	**Event**	**Next state**	**Expected result**
New Account	All required setup is completed	Active	Account is activated.
New Account	Required details are incomplete	New Account	Account remains pending.
Active	Suspicious activity is detected	Suspended	Restricted transactions are blocked and the event is recorded.
Suspended	Authorized review clears the issue	Active	Permitted operations are restored.
Active	Valid closure request and conditions are met	Closed	Account is closed and new transactions are blocked.
Suspended	Closure is permitted by policy and conditions are met	Closed	Account is closed through the approved process.
Closed	Attempt to transact or reactivate	Closed	Request is rejected; closed state remains.

Invalid transitions include Suspended → Active without authorization and Closed → Active. If policy forbids it, New Account → Closed without the required procedure is also invalid. Test repeated suspension events and simultaneous closure and transfer requests. For every test, verify the state, permitted operations, database record, audit log and notification. Closed should be treated as a terminal state unless requirements explicitly say otherwise.

Q11. State-Based Testing for a Food Delivery Application

The normal order flow is: Order Placed → Payment Confirmed → Restaurant Accepted → Food Preparing → Out for Delivery → Delivered. The order may also enter Payment Failed, Cancelled or Refund Initiated.

Current state	Event	Expected state / result
Order Placed	Payment succeeds	Payment Confirmed
Order Placed	Payment fails	Payment Failed; restaurant preparation must not start.
Payment Failed	Payment retry succeeds	Payment Confirmed without duplicate charge.
Payment Confirmed	Restaurant accepts order	Restaurant Accepted
Restaurant Accepted	Food preparation begins	Food Preparing
Food Preparing	Order is dispatched	Out for Delivery
Out for Delivery	Delivery is confirmed	Delivered
Eligible pre-delivery state	Valid cancellation request	Cancelled; refund initiated if payment was collected.
Cancelled	Refund processing starts	Refund Initiated
Delivered	Duplicate delivery event arrives	Remain Delivered; no duplicate completion.
Delivered	Late cancellation request arrives	Reject or send for support according to policy.

Invalid transitions include Delivered → Food Preparing and Payment Failed → Restaurant Accepted. Also test repeated payment callbacks, duplicate restaurant responses, delayed delivery updates, network loss and app restart. Recovery testing should confirm that retrying does not duplicate charges, order status remains consistent, refunds can be tracked and customers receive accurate updates.

Q12. Decision Table Testing for Insurance Claims

Decision Table Testing lists conditions and the action for each meaningful combination. For this example, a claim is approved only when the policy is valid, premium is paid, the accident is covered, documents are complete and the claim amount is within the policy limit.

Rule	Policy valid	Premium paid	Accident covered	Documents complete	Within limit	Action
R1	Yes	Yes	Yes	Yes	Yes	Approve claim
R2	No	—	—	—	—	Reject: invalid policy
R3	Yes	No	—	—	—	Reject or hold:

						premium unpaid
R4	Yes	Yes	No	—	—	Reject: event not covered
R5	Yes	Yes	Yes	No	—	Request documents / hold
R6	Yes	Yes	Yes	Yes	No	Refer for review or reject according to policy

A dash means that condition does not affect the rule once an earlier condition has already decided the outcome. Test one case for each rule: fully eligible claim, invalid policy, unpaid premium, excluded accident, missing documents and claim above the limit. Also test a claim exactly at the policy limit and just above it. The actual system must follow the insurer's approved rules.

Q13. Decision Table Testing for a Login System

The login succeeds only if the username is registered, the password is correct, the account is active, the account is not locked and CAPTCHA is verified.

Rule	Username registered	Password correct	Account active	Not locked	CAPTCHA verified	Expected result
R1	Yes	Yes	Yes	Yes	Yes	Login successful
R2	No	—	—	—	—	Deny login
R3	Yes	No	—	—	—	Deny login
R4	Yes	Yes	No	—	—	Deny login
R5	Yes	Yes	Yes	No	—	Deny login: locked account
R6	Yes	Yes	Yes	Yes	No	Deny login: CAPTCHA failed

These six tests form a minimum useful set: one test confirms successful login and five tests check each individual failure condition. Additional tests should cover several failures at once, expired CAPTCHA, repeated wrong passwords and account-locking limits. Error messages should not reveal unnecessary information about whether a username exists.

Q14. Black-Box Testing for Password Rules

A valid password must contain 8–20 characters, at least one uppercase letter, one lowercase letter, one digit and one special character. The exact rules for spaces and Unicode characters should be defined by the system requirements.

Test	Example/input	Expected result	Defect checked

Minimum boundary	Abcdef1! (8 characters)	Accept	Incorrect minimum length.
Below minimum	Abcde1! (7 characters)	Reject	Minimum length not enforced.
Maximum boundary	A password of exactly 20 characters containing all four required character types	Accept	Incorrect maximum length handling.
Above maximum	A password of 21 characters containing all required character types	Reject	Maximum length not enforced.
No uppercase	abcdefg1!	Reject	Uppercase rule missing.
No lowercase	ABCDEFG1!	Reject	Lowercase rule missing.
No digit	Abcdefgh!	Reject	Digit rule missing.
No special character	Abcdefg1	Reject	Special-character rule missing.
All rules satisfied	Secure9!a	Accept	Valid password wrongly rejected.
Empty password	Empty string	Reject	Required-field check missing.
Missing password field	No password value sent	Reject safely	Null or missing input not handled.
Spaces	Password with leading/trailing spaces	Follow documented policy	Inconsistent space handling.
Unexpected characters	Password containing Unicode or markup-like characters	Follow documented character policy; process safely	Character handling or security defect.
Very long input	Thousands of characters	Reject safely without crashing	Resource exhaustion or truncation.

Equivalence Partitioning tests representative valid and invalid password groups. Boundary Value Analysis checks lengths 7, 8, 9, 19, 20 and 21. Negative testing checks missing character types, blank input, unexpected characters and very long input. Use strings of verified lengths when executing the tests.

Q15. Static Black-Box Testing of Online Shopping Requirements

Static black-box testing reviews requirement documents without running the software. It checks whether requirements are clear, complete, consistent and testable before development starts.

Requirement example	**Problem type**	**Explanation / improvement**
“The website should load quickly.”	Ambiguous and unverifiable	Define a measurable response-time target and the load under which it must be met.
“Users can pay using available methods.”	Incomplete	List payment methods and define timeout, failure and refund behavior.
One section says discounts apply to eligible orders; another says all orders receive discounts.	Contradictory	Clarify eligibility and make both statements consistent.
Checkout does not explain what happens when an item becomes out	Missing	Define stock checking, reservation and the message shown to the

of stock.		customer.
“The system supports returns.”	Incomplete	Specify the return window, eligible products, refund method and exceptions.
“Only authorized users can view orders.”	Not sufficiently testable	Define roles, order ownership checks and admin permissions.
No rule covers payment timeout or duplicate payment notification.	Missing	Specify retries, duplicate prevention, transaction status and reconciliation.
“Search results should be relevant.”	Ambiguous	Define searchable fields, sorting, filters and behavior when no results exist.
Shipping charges differ between the cart and checkout requirements.	Contradictory	Define one consistent shipping-charge rule.
No privacy or data-retention requirements are included.	Missing	Specify how personal information is protected and how long it is retained.

Review checklist before implementation

- Each requirement has a unique ID and clear purpose.
- Requirements use clear words and avoid vague terms without measurable definitions.
- Acceptance criteria are measurable and testable.
- User roles and permissions are defined.
- Product search, filters, cart, stock and checkout are covered.
- Shipping, taxes, discounts, payment, cancellation, returns and refunds are explained.
- Normal cases, invalid inputs, boundaries, timeouts and recovery are specified.
- External services and their error responses are documented.
- Security and privacy requirements are included.
- Performance, availability, accessibility and compatibility targets are measurable.
- Required fields, formats, data types and value limits are defined.
- Contradictions and duplicated requirements are resolved.
- Assumptions, dependencies and unanswered questions are recorded.
- Requirements are prioritized according to risk and business importance.
- Each requirement can be linked to a test case or review check.

The review team should record each requirement defect, ask questions, update the document and confirm the corrections before coding begins. This reduces rework and gives testers a clear basis for preparing test cases.